

Strumenti

Buone pratiche

Regole semplici per scrivere codice Java piu leggibile e mantenibile.

Scrivere per farsi capire

Il codice viene letto molte piu volte di quante venga scritto.

Anche quando lavori da solo, il "lettore futuro" sarai tu fra qualche settimana.

Le buone pratiche servono a rendere il codice piu chiaro, non a renderlo piu elegante per forza.

Usa nomi chiari

Meglio:

```
double prezzoTotale = prezzo + spedizione;
```

che:

```
double x = a + b;
```

Nomi come `x`, `a`, `b` vanno bene solo in esempi molto piccoli o calcoli evidenti.

Per variabili e metodi usa il camelCase:

```
nomeUtente  
calcolaTotale()  
mostraMessaggio()
```

Per classi usa la maiuscola iniziale:

```
Persona  
Prodotto  
Calcolatrice
```

Metodi piccoli

Un metodo dovrebbe fare una cosa principale.

Se un metodo:

- legge input
- calcola
- stampa
- salva su file

tutto insieme, diventa difficile da capire e da testare.

Prova a dividerlo:

```
public static double calcolaTotale(double prezzo, int quantita) {  
    return prezzo * quantita;  
}  
  
public static void mostraTotale(double totale) {  
    System.out.println("Totale: " + totale);  
}
```

Evita duplicazioni

Se ripeti lo stesso blocco piu volte, valuta un metodo.

Prima:

```
System.out.println("-----");
System.out.println("Prodotti");
System.out.println("-----");
System.out.println("Totale");
System.out.println("-----");
```

Dopo:

```
public static void separatore() {
    System.out.println("-----");
}
```

La duplicazione non e solo una questione di righe. Se devi cambiare qualcosa, rischi di dimenticare una copia.

Formatta il codice

Usa indentazione coerente.

```
if (eta >= 18) {
    System.out.println("Maggiorenne");
} else {
    System.out.println("Minorenne");
}
```

Evita codice tutto attaccato:

```
if(eta>=18){System.out.println("Maggiorenne");}
```

Il computer lo capisce, ma le persone fanno piu fatica.

Non ottimizzare troppo presto

All'inizio scrivi codice chiaro.

Non cercare la versione piu corta o piu furba se rende il programma difficile da leggere.

Prima chiediti:

- funziona?
- e chiaro?
- e facile da modificare?

Solo dopo ha senso pensare alle prestazioni, se c'e un problema reale.

Commenti utili

Un commento deve spiegare cio che il codice non rende evidente.

Utile:

```
// Applichiamo lo sconto solo agli ordini sopra 50 euro
if (totale > 50) {
    totale = totale - 5;
}
```

Poco utile:

```
// Aggiunge 5 a totale
totale = totale + 5;
```

Regola pratica

Quando finisci un programma, rileggilo come se fosse di un'altra persona.

Se una riga ti costringe a fermarti troppo, forse puoi:

- rinominare una variabile
- estrarre un metodo
- aggiungere una nota breve
- dividere un'espressione in due passaggi

Il codice buono non e quello che sembra difficile. E quello che puoi capire, correggere e far crescere.

Maven e Gradle

A cosa servono gli strumenti di build nei progetti Java.

Perche servono strumenti di build

Nei primi esercizi compili con:

```
javac NomeFile.java
```

Funziona bene per programmi piccoli.

Nei progetti reali ci sono molte classi, librerie esterne, test e file di configurazione. Gestire tutto a mano diventa scomodo.

Qui entrano in gioco Maven e Gradle.

Cosa fanno Maven e Gradle

Uno strumento di build puo:

- compilare il progetto
- scaricare librerie esterne
- eseguire test
- creare un file pronto da distribuire
- mantenere una struttura ordinata

Le librerie esterne si chiamano spesso **dipendenze**: pezzi di codice scritti da altri che il tuo progetto usa.

Maven

Maven usa un file chiamato `pom.xml`.

Una struttura tipica e:

```
progetto/  
  pom.xml  
  src/  
    main/  
      java/  
    test/  
      java/
```

Il codice principale va in:

```
src/main/java
```

I test vanno in:

```
src/test/java
```

Comandi comuni:

```
mvn compile  
mvn test  
mvn package
```

Gradle

Gradle usa spesso un file `build.gradle` oppure `build.gradle.kts` .

Anche Gradle usa una struttura simile:

```
progetto/  
  build.gradle  
  src/  
    main/  
      java/  
    test/  
      java/
```

Comandi comuni:

```
gradle build  
gradle test
```

In molti progetti troverai uno script incluso:

```
./gradlew build
```

Su Windows:

```
gradlew.bat build
```

Questo permette di usare Gradle senza installarlo manualmente.

Dipendenze

Se vuoi usare una libreria esterna, la dichiari nel file di build.

Non devi scaricare manualmente file `.jar` e copiarli in giro.

Lo strumento di build legge la configurazione, scarica ciò che serve e lo collega al progetto.

Quando ti servono davvero

All'inizio puoi continuare con `javac` e `java`.

Maven o Gradle diventano utili quando:

- il progetto ha molte classi
- vuoi usare librerie esterne
- vuoi scrivere test automatici
- lavori con altre persone
- usi framework come Spring

Regola pratica

Per imparare la sintassi Java, parti semplice.

Quando il programma inizia ad avere piu cartelle, librerie e test, passa a Maven o Gradle.

Non sono "Java avanzato" nel senso del linguaggio, ma sono parte della vita quotidiana di molti progetti Java.

Package e import

Come organizzare classi in package e usare codice da altre librerie.

Perche organizzare il codice

Quando un progetto contiene poche classi, puoi tenerle nella stessa cartella.

Quando cresce, serve ordine. I **package** aiutano a raggruppare classi collegate.

Un package e come una cartella logica per il codice.

Dichiarare un package

In cima a un file Java puoi scrivere:

```
package negozio;
```

Poi definisci la classe:

```
package negozio;

public class Prodotto {
    private String nome;

    public Prodotto(String nome) {
        this.nome = nome;
    }

    public String getNome() {
        return nome;
    }
}
```

Di solito la cartella rispecchia il package:

```
negozio/Prodotto.java
```

Usare import

Se una classe si trova in un package diverso, puoi importarla.

Questo import ti permette di scrivere:

```
ArrayList nomi = new ArrayList<>();
```

Senza import dovresti scrivere il nome completo:

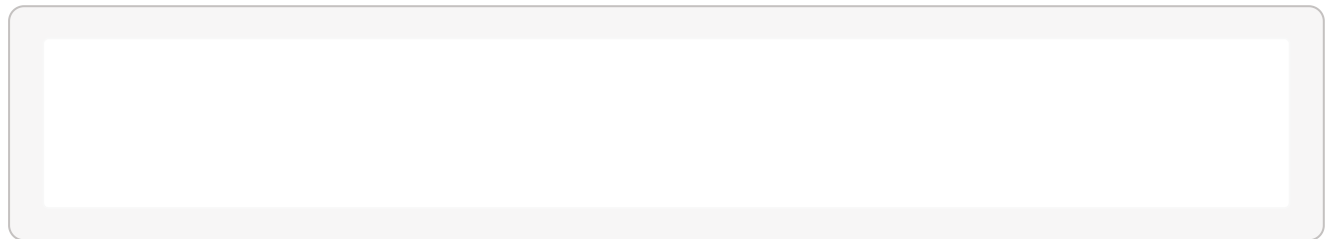
```
java.util.ArrayList nomi = new java.util.ArrayList<>();
```

Troppo lungo per l'uso quotidiano.

Import dalla libreria standard

Molte classi utili fanno parte della libreria standard Java.

Esempi:



`java.util` contiene molte strutture e strumenti generali.

`java.io` contiene classi per input e output, inclusi i file.

Importare una propria classe

Immagina questa struttura:

```
src/  
  negozio/  
    Prodotto.java  
  app/  
    Main.java
```

`Prodotto.java` :

```
package negozio;  
  
public class Prodotto {  
}
```

`Main.java` :

```
package app;

public class Main {
    public static void main(String[] args) {
        Prodotto prodotto = new Prodotto();
    }
}
```

Errori comuni

Il package dichiarato deve essere coerente con la cartella.

Se un file dice:

```
package negozio;
```

ma lo tieni in una cartella non coerente, strumenti e compilatore possono confondersi.

Un altro errore comune è dimenticare `public` su una classe che vuoi usare da un altro package.

```
public class Prodotto {
}
```

Regola pratica

All'inizio usa package semplici e nomi chiari.

Quando un progetto cresce, raggruppa per argomento:

```
app
model
service
repository
```

Non creare package solo per complicare. Devono aiutare a trovare il codice.

Test

Come verificare il comportamento del codice con test automatici semplici.

Perche scrivere test

Un test automatico controlla che una parte del programma funzioni come previsto.

Invece di provare tutto a mano ogni volta, scrivi codice che verifica altro codice.

Questo e utile quando:

- modifichi una funzione
- vuoi evitare regressioni
- lavori su programmi piu grandi

Un metodo da testare

Immagina questa classe:

```
public class Calcolatrice {  
    public static int somma(int a, int b) {  
        return a + b;  
    }  
}
```

Vogliamo controllare che `somma(2, 3)` restituisca `5`.

L'idea di test unitario

Un **test unitario** controlla una piccola unita di codice, spesso un metodo.

Con JUnit, una libreria molto usata per i test Java, potresti scrivere:

```
public class CalcolatriceTest {  
    @Test  
    void sommaDueNumeri() {  
        int risultato = Calcolatrice.somma(2, 3);  
        assertEquals(5, risultato);  
    }  
}
```

`assertEquals(5, risultato)` significa: mi aspetto che `risultato` sia uguale a `5`.

Test che passa e test che fallisce

Se il metodo è corretto:

```
return a + b;
```

il test passa.

Se per errore scrivi:

```
return a - b;
```

il test fallisce e ti segnala che il risultato ottenuto non corrisponde a quello atteso.

Cosa testare

Inizia da metodi semplici che restituiscono un valore.

Esempi buoni:

```
calcolaTotale(2.50, 4)
isMaggiorenni(20)
formattaNome("luca")
```

Sono facili da controllare perché hanno input chiari e output chiari.

Evitare test troppo grandi

Un test dovrebbe controllare una cosa principale.

Meglio tre test piccoli:

- somma numeri positivi
- somma con zero
- somma numeri negativi

che un solo test lungo che controlla tutto insieme.

Test e strumenti di build

JUnit di solito si usa dentro un progetto Maven o Gradle.

Con Maven, il comando tipico è:

```
mvn test
```

Con Gradle:

```
gradle test
```

Questi strumenti compilano il codice, eseguono i test e mostrano il risultato.

Regola pratica

All'inizio non devi testare ogni riga.

Prova a testare i metodi che:

- fanno calcoli
- controllano condizioni
- trasformano dati
- possono rompersi quando li modifichi

I test non sostituiscono il ragionamento, ma ti danno una rete di sicurezza.